Deep Residual Learning for Image Recognition

2019.09.20. Fri.

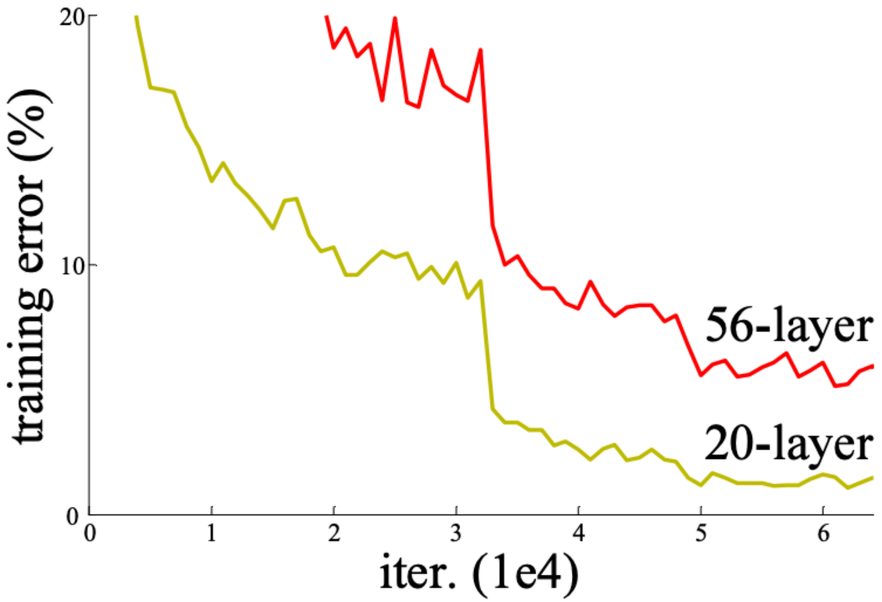
Member: 고진호, DaWoon-Heo

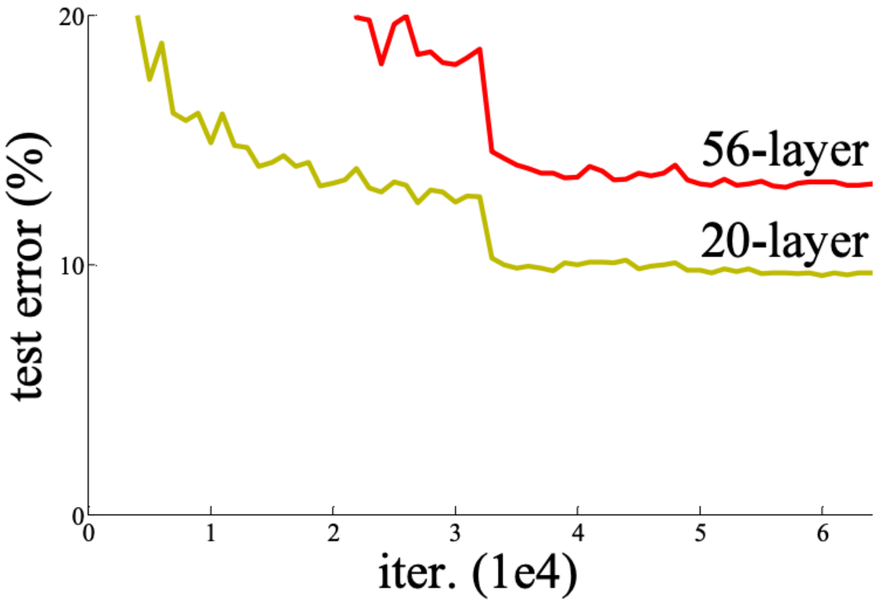
Presenter: 고진호

CONTENTS

1. Introduction

- Overfitting vs. Degradation
  - Overfitting: occurs due to the data (training data excessively fit to the model)
  - Degradation: occurs when the networks getting deeper (performance issue)





2. Related Work

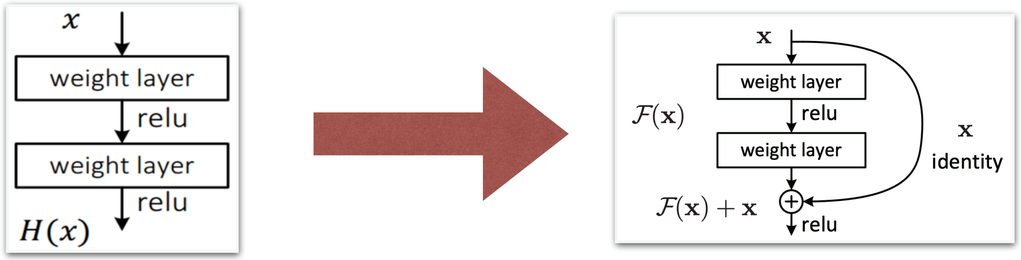
- Residual Representation
  - Vector of Locally Aggregated Descriptors (VLAD)
  - Fisher Vector - probabilistic version of Vlad
  - Partial Differential Equations (PDEs)

- Shortcut Connection
  - Highway networks - not always learns residual functions

    (A gated shortcut is “closed”, the layers in highway networks represent non-residual functions)

3. Deep Residual Learning

- ‘Residual’의 의미는 ‘남은, 잔여의’, 수학적으로 예측값과 실제값의 차이를 의미하기도 함
- Original mapping 보다 residual mapping 이 더 최적하시키기 좋을것이라는 직관에서 시작



- Original mapping을 H(x)로 정의/ H(x) := F(x) + x
- H(x)값에 초점을 맞추기보다 ‘F(x) = H(x) - x’라는 점에 초점을 맞추는 모델
- 이전에 누적된 데이터를 학습하지않고 그대로 출력

3. Deep Residual Learning

- Identity shortcut

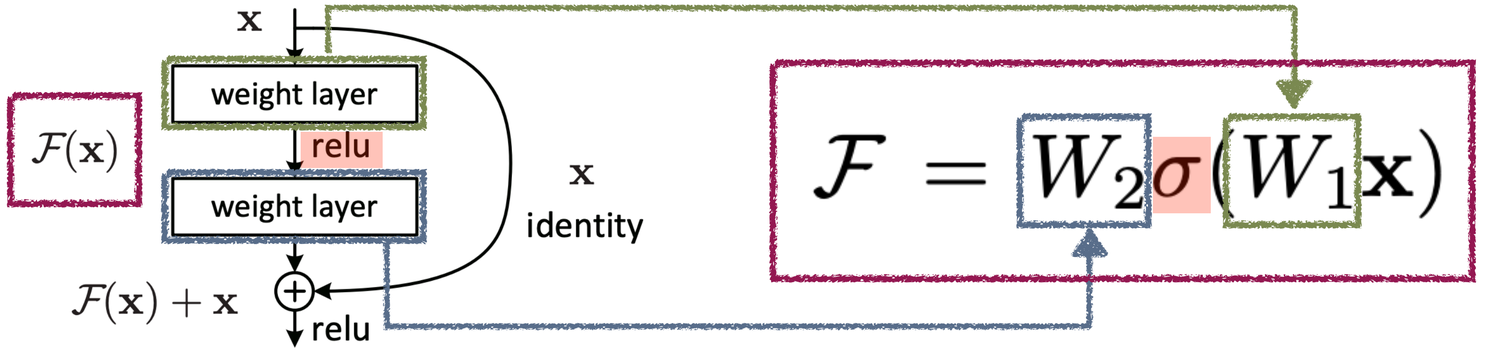
$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}.$$

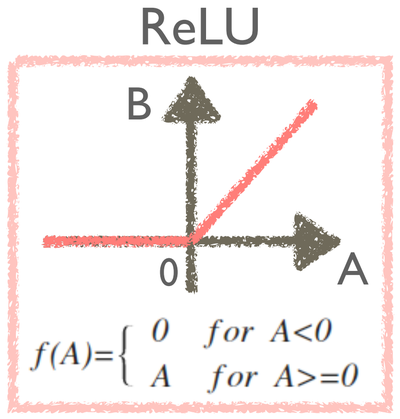
  - y: output vector of the layer
  - x: input (identity) vector of the layer
  - W: weight

- Projection shortcut

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + W_s\mathbf{x}$$

  - y: output vector of the layer
  - x: input vector (identity) of the layer
  - W: weight
  - s: square matrix

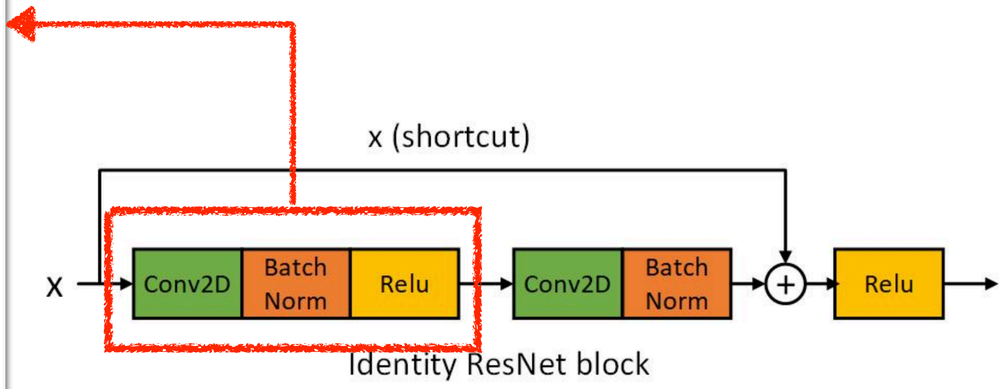




3. Deep Residual Learning

- **Identity block**

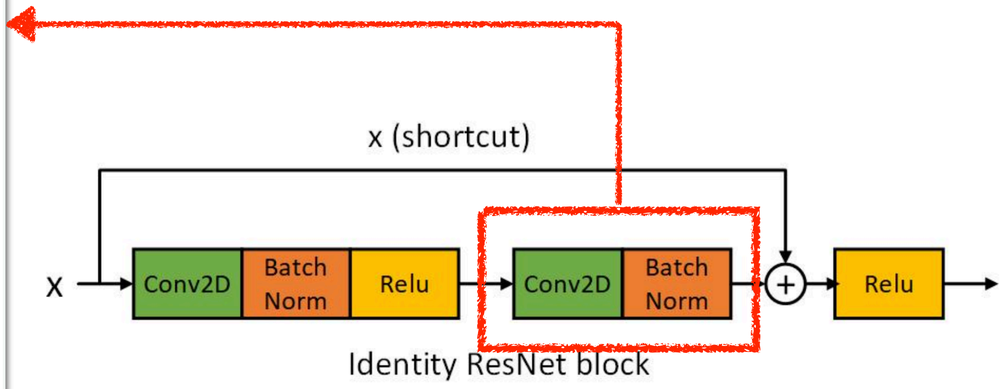
```
x = layers.Conv2D(
    filters1, (1, 1),
    use_bias=False,
    kernel_initializer='he_normal',
    kernel_regularizer=_gen_l2_regularizer(use_l2_regularizer),
    name=conv_name_base + '2a')(
        input_tensor)
x = layers.BatchNormalization(
    axis=bn_axis,
    momentum=BATCH_NORM_DECAY,
    epsilon=BATCH_NORM_EPSILON,
    name=bn_name_base + '2a')(
        x)
x = layers.Activation('relu')(x)
```



3. Deep Residual Learning

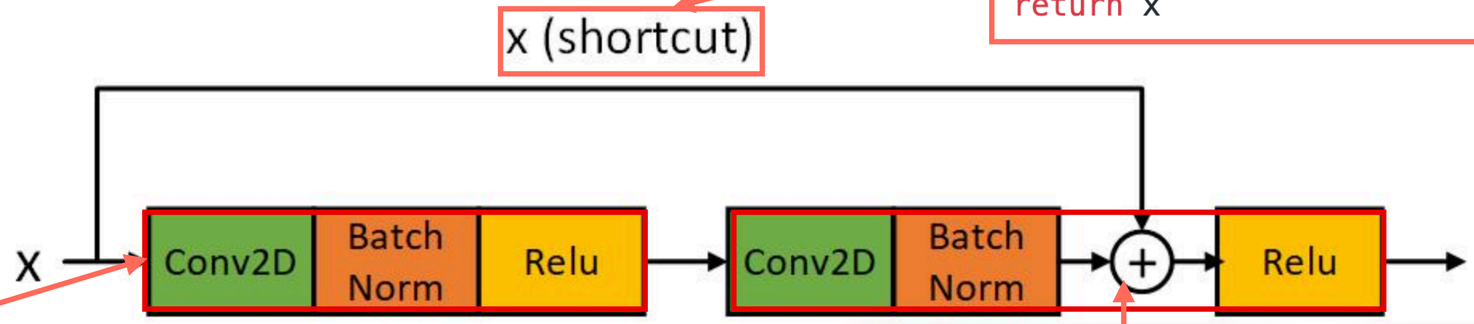
- **Identity block**

```
x = layers.Conv2D(
    filters2,
    kernel_size,
    padding='same',
    use_bias=False,
    kernel_initializer='he_normal',
    kernel_regularizer=_gen_l2_regularizer(use_l2_regularizer),
    name=conv_name_base + '2b')(
        x)
x = layers.BatchNormalization(
    axis=bn_axis,
    momentum=BATCH_NORM_DECAY,
    epsilon=BATCH_NORM_EPSILON,
    name=bn_name_base + '2b')(
        x)
```



3. Deep Residual Learning

- **Identity block**



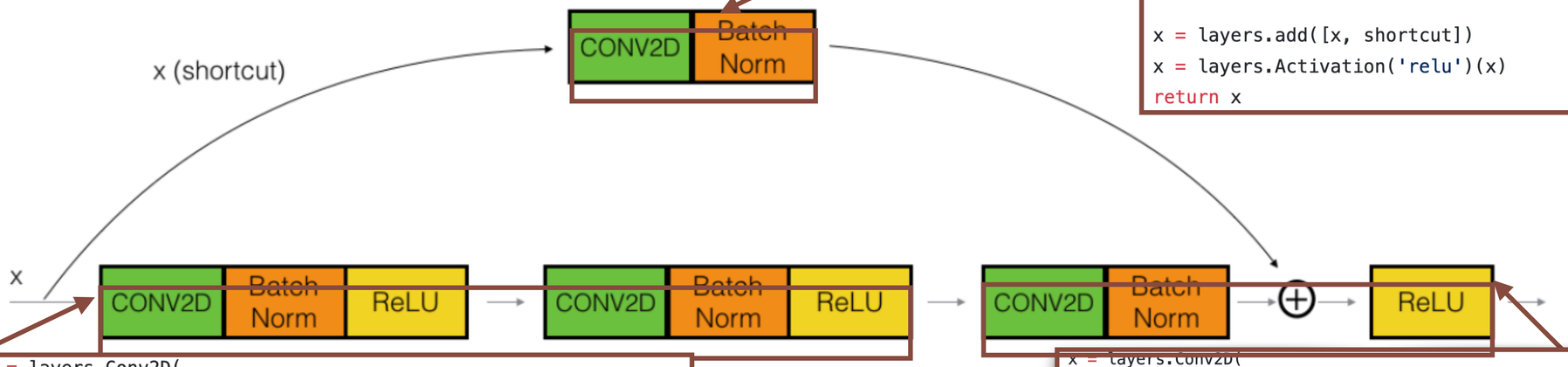

```
x = layers.add([x, input_tensor])
x = layers.Activation('relu')(x)
return x
```

```
x = layers.Conv2D(
    filters1, (1, 1),
    use_bias=False,
    kernel_initializer='he_normal',
    kernel_regularizer=_gen_l2_regularizer(use_l2_regularizer),
    name=conv_name_base + '2a')(
        input_tensor)
x = layers.BatchNormalization(
    axis=bn_axis,
    momentum=BATCH_NORM_DECAY,
    epsilon=BATCH_NORM_EPSILON,
    name=bn_name_base + '2a')(
        x)
x = layers.Activation('relu')(x)
```

```
x = layers.Conv2D(
    filters2,
    kernel_size,
    padding='same',
    use_bias=False,
    kernel_initializer='he_normal',
    kernel_regularizer=_gen_l2_regularizer(use_l2_regularizer),
    name=conv_name_base + '2b')(
        x)
x = layers.BatchNormalization(
    axis=bn_axis,
    momentum=BATCH_NORM_DECAY,
    epsilon=BATCH_NORM_EPSILON,
    name=bn_name_base + '2b')(
        x)
x = layers.Activation('relu')(x)
```

3. Deep Residual Learning

- **Convolution block**




```
shortcut = layers.Conv2D(
    filters3, (1, 1),
    strides=strides,
    use_bias=False,
    kernel_initializer='he_normal',
    kernel_regularizer=_gen_l2_regularizer(use_l2_regularizer),
    name=conv_name_base + '1')(
        input_tensor)
shortcut = layers.BatchNormalization(
    axis=bn_axis,
    momentum=BATCH_NORM_DECAY,
    epsilon=BATCH_NORM_EPSILON,
    name=bn_name_base + '1')(
        shortcut)

x = layers.add([x, shortcut])
x = layers.Activation('relu')(x)
return x
```

```
x = layers.Conv2D(
    filters1, (1, 1),
    use_bias=False,
    kernel_initializer='he_normal',
    kernel_regularizer=_gen_l2_regularizer(use_l2_regularizer),
    name=conv_name_base + '2a')(
        input_tensor)
x = layers.BatchNormalization(
    axis=bn_axis,
    momentum=BATCH_NORM_DECAY,
    epsilon=BATCH_NORM_EPSILON,
    name=bn_name_base + '2a')(
        x)
x = layers.Activation('relu')(x)
```

```
x = layers.Conv2D(
    filters2,
    kernel_size,
    padding='same',
    use_bias=False,
    kernel_initializer='he_normal',
    kernel_regularizer=_gen_l2_regularizer(use_l2_regularizer),
    name=conv_name_base + '2b')(
        x)
x = layers.BatchNormalization(
    axis=bn_axis,
    momentum=BATCH_NORM_DECAY,
    epsilon=BATCH_NORM_EPSILON,
    name=bn_name_base + '2b')(
        x)
x = layers.Activation('relu')(x)
```

4. Experiments ImageNet Classification

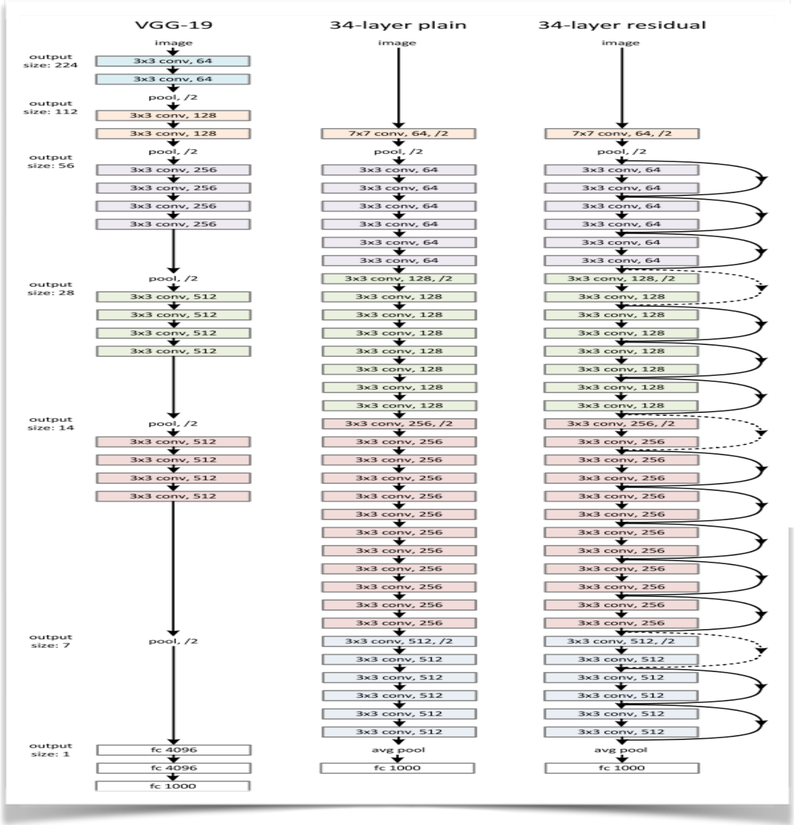
- **ImageNet dataset**

	NUMBER OF CLASS	NUMBER OF TRAINING IMAGES	NUMBER OF VALIDATION IMAGES	NUMBER OF TEST IMAGES
	1,000	1,280,000	50,000	100,000

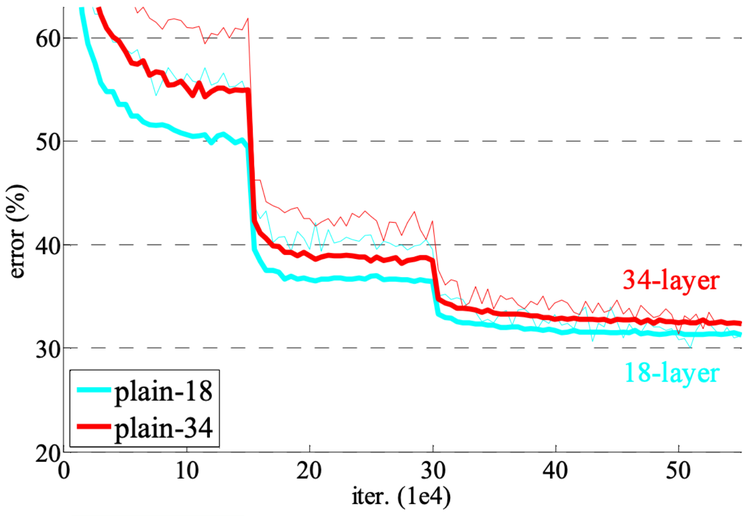
- **ImageNet Classification에서는 3가지 포인트에 대해서 실험을 진행**
  - Comparing Plain network and ResNet
  - Identity and projection shortcut impacts on the performance
  - Comparing 34, 50, 101 and 152 layer ResNets

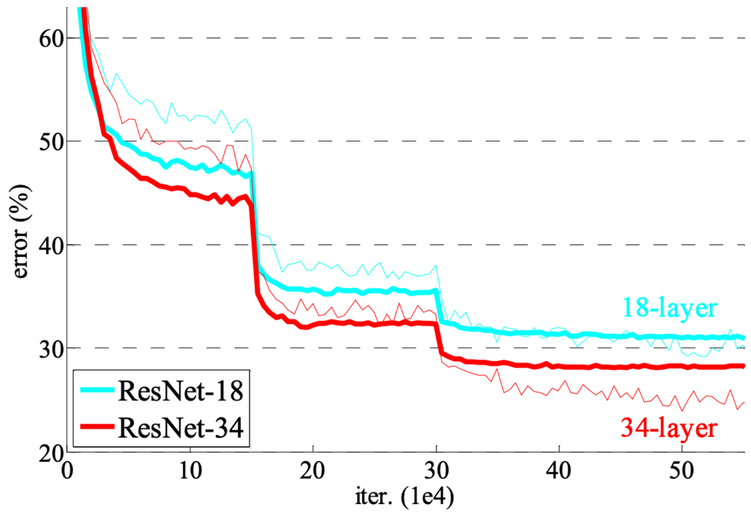
4. Experiments ImageNet Classification

- **Comparing Plain network and ResNet**



<Architecture>





	plain	ResNet
18 layers	27.94	27.88
34 layers	28.54	**25.03**

<Result>

4. Experiments ImageNet Classification

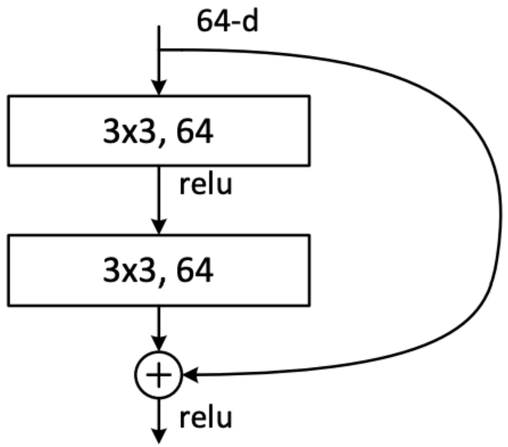
- **Identity and projection shortcut impacts on the performance**

	RESNET-A	RESNET-B	RESNET-C
INCREASING DIMENSIONS	Zero-padding shortcut	Projection shortcut	Projection shortcut
NORMAL	Identity shortcut	Identity shortcut	Projection shortcut
ERROR RATE TOP1	25.03	24.52	24.19
ERROR RATE TOP5	7.76	7.46	7.40

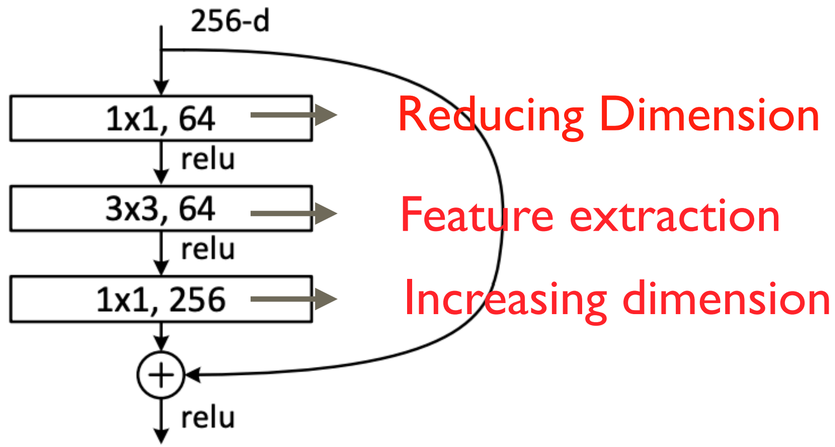
- Resnet-C가 가장 좋은 퍼포먼스를 보여주지만 그 차이가 미세하고 memory/time complexity가 늘어나기 때문에 효율적이지 않다.

4. Experiments ImageNet Classification

- **Comparing 34, 50, 101 and 152 layer ResNets**



<Building block>



<Bottleneck block>

- ResNet34는 Building block을 사용, 반면, ResNet50, 101 그리고 152는 Bottleneck block을 사용하여 더 깊은 구조를 만듬.

4. Experiments ImageNet Classification

- **Comparing 34, 50, 101 and 152 layer ResNets**

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$
conv3_x	28×28	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times8$
conv4_x	14×14	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times23$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times36$
conv5_x	7×7	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

<Architecture>

method	top-1 err.	top-5 err.
ResNet-34 B	21.84	5.71
ResNet-34 C	21.53	5.60
ResNet-50	20.74	5.25
ResNet-101	19.87	4.60
ResNet-152	**19.38**	**4.49**

<Result> Error rates (%)

5. Experiments CIFAR-10

- **CIFAR-10 dataset**

	NUMBER OF CLASS	NUMBER OF TRAINING IMAGES	SIZE OF IMAGE	NUMBER OF TEST IMAGES
	10	50,000	32×32	10,000

- **CIFAR-10 Classification는 보다 훨씬 더 깊은 네트워크에서 ResNet이 error rate를 줄여줄수 있는지를 확인하기위해 실험을 진행**

5. Experiments CIFAR-10

- 하지만 Resnet에서 layers을 더 많이 쌓았을 경우(1000이상)에는
- Overfittting 문제를 일으킨다.
- 이 논문에서는 maxout이나 dropout등의 정규화를 쓰지않았다.
- 정규화를 통해 더 좋은 결과를 기대할수 있다.